

M05 / PROTECTED ANSWERS

M05 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
A01 - DE05-01 CSV properly: parse, inspect, preserve raw	3
A02 - DE05-02 JSON and JSONL from zero	3
A03 - DE05-03 Paths, folders and deterministic batch files	4
A04 - DE05-04 pandas profiling before transformation	4
A05 - DE05-05 Cleaning with pandas and rejected rows	5
A06 - DE05-06 pandas merge, cardinality and aggregation	5
A07 - DE05-07 Data validation: record, batch and relationship rules	6
A08 - DE05-08 HTTP and REST APIs from zero	6
A09 - DE05-09 Calling APIs safely with requests	7
A10 - DE05-10 Pagination and authentication without leaking secrets	7
A11 - DE05-11 Retries, rate limits and partial failure	8
A12 - DE05-12 Python + PostgreSQL with SQLAlchemy	8
A13 - DE05-13 Configuration, environment variables and secret hygiene	9
A14 - DE05-14 Logging, testing and reusable pipeline code	9
A15 - DE05-15 Integrated Python data-engineering pipeline	10

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a saved closed-book attempt. Each answer gives a concrete solution, a proof path, common mistakes, validation, and a changed retry. Do not overwrite your first attempt.

A01 - DE05-01 | CSV properly: parse, inspect, preserve raw

One acceptable solved answer

Use a real CSV parser, keep identifiers as strings, and fingerprint the untouched bytes before transformation. A quoted field such as "Delhi, NCR" must remain one field; 0009 must remain the string 0009.

Worked proof / example

```
from pathlib import Path
import csv, hashlib
raw=Path('orders.csv').read_bytes()
print(hashlib.sha256(raw).hexdigest())
with open('orders.csv', newline='', encoding='utf-8') as f:
    rows=list(csv.DictReader(f))
assert rows[0]['city']=='Delhi, NCR'
assert rows[0]['id']=='0009'
```

Common beginner mistakes

Using `split(',')` breaks quoted commas; coercing the ID to int destroys leading zeros; hashing the cleaned DataFrame does not prove the source bytes were preserved.

Validate your own attempt

Record header count, record count, the quoted city value, the exact ID string and the raw SHA-256. Reopen the source and prove it was not edited.

Changed retry

Create a second CSV with an embedded quote and an empty optional field. Parse it without changing grain or identifier text.

A02 - DE05-02 | JSON and JSONL from zero

One acceptable solved answer

Treat a JSON document and JSONL stream as different grains. In JSONL, parse each physical line independently. Optional nested metadata can be absent on one event without forcing a fake empty object into every other event.

Worked proof / example

```
import json
with open('events.jsonl', encoding='utf-8') as f:
    events=[json.loads(line) for line in f if line.strip()]
assert len(events)==3
assert 'metadata' in events[1]
assert 'metadata' not in events[0]
```

Common beginner mistakes

Loading the entire JSONL file with `json.load()`; treating null and missing as identical without a contract; flattening nested structures before preserving raw evidence.

Validate your own attempt

Prove three independent events were read and explain the difference between missing key, null value and empty object/list.

Changed retry

Make event 3 contain an optional list of tags; preserve compatibility without changing event grain.

A03 - DE05-03 | Paths, folders and deterministic batch files

One acceptable solved answer

Choose an explicit ordering rule. If `business_date` controls processing, read each candidate metadata/date, sort by (`business_date`, stable filename) and never rely on directory iteration order. Resolve paths from a deliberate project/input root rather than the current terminal directory.

Worked proof / example

```
from pathlib import Path
files=list(Path('input').glob('*.csv'))
# after reading business_date metadata:
ordered=sorted(files, key=lambda p:(business_date[p.name], p.name))
```

Common beginner mistakes

Assuming lexical filename order equals business order; using `os.listdir()` order as a contract; silent output overwrite.

Validate your own attempt

Run from two different working directories and prove the same input set/order and fingerprints are selected.

Changed retry

Use filenames Z.csv and A.csv where business dates require A after Z; prove the business rule wins.

A04 - DE05-04 | pandas profiling before transformation

One acceptable solved answer

Profile the untouched raw population first: row count, columns/dtypes, null counts, duplicate-key counts and suspicious numeric/date parse candidates. Do not silently clean while profiling.

Worked proof / example

```
import pandas as pd
df=pd.read_csv('inventory.csv', dtype={'sku':'string'})
print(len(df))
print(df.isna().sum())
```

```
print(df['sku'].duplicated(keep=False).sum())
print(pd.to_numeric(df['quantity'], errors='coerce').isna().sum())
```

Common beginner mistakes

Dropping bad rows before reporting the raw condition; using describe() alone; treating dtype object as proof values are valid.

Validate your own attempt

Your raw count must still equal the source count. Save profiling observations separately from cleaned output.

Changed retry

Add one blank numeric value and one duplicate key; report both before changing anything.

A05 - DE05-05 | Cleaning with pandas and rejected rows

One acceptable solved answer

Split accepted and rejected rows with an explicit reason. For the fresh inventory contract: SKU unique, quantity positive integer, cost finite/nonnegative decimal, received_date ISO. Never make invalid rows disappear.

Worked proof / example

```
valid_qty = pd.to_numeric(df.quantity, errors='coerce').notna() & (pd.to_numeric(df.quantity, errors='coerce')>0)
valid_cost = pd.to_numeric(df.unit_cost, errors='coerce').notna() & (pd.to_numeric(df.unit_cost, errors='coerce')>=0)
dup = df.sku.duplicated(keep=False)
accepted=df[valid_qty & valid_cost & ~dup].copy()
rejected=df[~(valid_qty & valid_cost & ~dup)].copy()
assert len(df)==len(accepted)+len(rejected)
```

Common beginner mistakes

keep="first" on duplicates without an authority rule; coercing failures to NaN then dropping them; not retaining rejection reason/raw identity.

Validate your own attempt

Reconcile raw = accepted + rejected and independently recompute an accepted total.

Changed retry

Introduce a negative cost and duplicate SKU; prove both are rejected with different reasons.

A06 - DE05-06 | pandas merge, cardinality and aggregation

One acceptable solved answer

The product/customer dimension must be unique on the lookup key before a many-to-one merge. Use pandas validation so fan-out fails loudly. Repair the duplicate dimension, then reconcile row count and amount before/after the join.

Worked proof / example

```
merged=fact.merge(dim, on='product_id', how='left', validate='many_to_one', indicator=True)
assert (merged['_merge']=='both').all()
assert len(merged)==len(fact)
assert merged.amount.sum()==fact.amount.sum()
```

Common beginner mistakes

Allowing a duplicate dimension to multiply fact rows and then aggregating away the evidence; checking only final totals.

Validate your own attempt

Prove key uniqueness in dim, fact row preservation, match membership and amount preservation.

Changed retry

Create one duplicate product key, observe MergeError, repair it and rerun the same controls.

A07 - DE05-07 | Data validation: record, batch and relationship rules

One acceptable solved answer

Separate scopes: record validation checks one row shape/type/range; batch validation checks uniqueness/reconciliation; relationship validation checks references across datasets. A syntactically valid customer_id can pass record checks and still fail membership against the dimension.

Worked proof / example

```
record_ok = isinstance(order['customer_id'], str) and bool(order['customer_id'])
relationship_ok = order['customer_id'] in set(customers.customer_id)
assert record_ok and not relationship_ok
```

Common beginner mistakes

Assuming Pydantic/row schema proves foreign keys or batch uniqueness; collapsing all failures into “invalid”.

Validate your own attempt

Show the same row PASS at record level and FAIL at relationship level with an understandable reason.

Changed retry

Add a duplicated valid order_id: record validation passes, batch uniqueness fails.

A08 - DE05-08 | HTTP and REST APIs from zero

One acceptable solved answer

HTTP transport success is not payload-contract success. A 200 response can still be unusable if content type is wrong or a required field is absent. Validate status policy, media type and payload shape separately.

Worked proof / example

```
r=requests.get(url, timeout=10)
r.raise_for_status()
if 'application/json' not in r.headers.get('content-type', ''):
    raise ValueError('expected JSON')
data=r.json()
if 'orders' not in data:
```

```
raise ValueError('missing orders')
```

Common beginner mistakes

Calling `r.json()` before checking status/content type; interpreting every 200 as usable data.

Validate your own attempt

Demonstrate one 200/missing-field case that is classified as contract failure, not transport failure.

Changed retry

Return 200 text/plain from a JSON endpoint and classify the failure correctly.

A09 - DE05-09 | Calling APIs safely with requests

One acceptable solved answer

For a 404: it is a client/application HTTP error. Do not parse it as successful JSON business data and do not blindly retry it as transient server/network failure.

Worked proof / example

```
try:
    r=requests.get(url, timeout=(3,10))
    r.raise_for_status()
except requests.HTTPError as e:
    if e.response.status_code==404:
        print('permanent/client-path error')
        raise
```

Common beginner mistakes

Retrying every exception; catching Exception and returning an empty list; losing status code and URL evidence.

Validate your own attempt

Saved evidence should show 404 classification and zero target writes/state advance.

Changed retry

Compare classification for connection refusal, 429, 500 and 404.

A10 - DE05-10 | Pagination and authentication without leaking secrets

One acceptable solved answer

Fetch until the API says there are no more pages (or until total is reached), accumulate by stable ID, validate total/uniqueness, and keep Authorization out of logs. Page size or request order must not change the logical set under a controlled replay.

Worked proof / example

```
seen={}
for page in pages:
    for row in fetch(page, token=os.environ['API_TOKEN']):
        if row['id'] in seen: raise ValueError('duplicate id')
        seen[row['id']]=row
```

```
assert len(seen)==expected_total
```

Common beginner mistakes

Stopping after page 1; logging headers/tokens; trusting page count without validating IDs/total.

Validate your own attempt

For the seven-record simulator, fetched count = unique ID count = declared total = 7 at two page sizes.

Changed retry

Use page size 3 and request pages in a controlled different order; final ID set must match.

A11 - DE05-11 | Retries, rate limits and partial failure

One acceptable solved answer

Use bounded retries only for retryable classes such as selected 5xx/429/network errors. Respect Retry-After where relevant. Authentication/validation failures are not made correct by retrying. With budget 2, final evidence must show attempt count, last status/error and no success/state advance.

Worked proof / example

```
for attempt in range(1,3):
    try:
        return call()
    except RetryableError:
        if attempt==2: raise
        time.sleep(backoff(attempt))
```

Common beginner mistakes

Infinite loops; retrying 401/404 blindly; writing partial target state before final success.

Validate your own attempt

Prove the flaky endpoint succeeds only within the budget or fails explicitly after the last allowed attempt.

Changed retry

Make the service fail three times with budget two; prove final failure is visible and bounded.

A12 - DE05-12 | Python + PostgreSQL with SQLAlchemy

One acceptable solved answer

Use one transaction for the two-row insert. Deliberately insert a duplicate unique key so the transaction raises and rolls back; neither row from that transaction should become durable.

Worked proof / example

```
from sqlalchemy import text
try:
    with engine.begin() as conn:
        conn.execute(text('insert into demo(id,v) values (:id,:v)'), rows)
except Exception:
```

```
pass
# query afterwards: zero rows from failed transaction
```

Common beginner mistakes

Autocommitting each row separately; catching the DB error and then claiming the transaction succeeded.

Validate your own attempt

Query the target after failure and prove the before-count/value set is unchanged.

Changed retry

Use a fresh two-row batch with a duplicate second key; then repair the key and prove both commit together.

A13 - DE05-13 | Configuration, environment variables and secret hygiene

One acceptable solved answer

Read input/output configuration from environment (or a deliberate config layer) before work begins. Required variables must fail fast. Secret values must not be hard-coded or printed.

Worked proof / example

```
from pathlib import Path
import os
input_dir=os.environ.get('INPUT_DIR')
if not input_dir: raise RuntimeError('INPUT_DIR required')
files=sorted(Path(input_dir).glob('*.csv'))
```

Common beginner mistakes

Falling back silently to a personal absolute path; printing secret values; creating outputs before discovering configuration is missing.

Validate your own attempt

Run once with changed INPUT_DIR without code edits; then unset it and prove failure happens before output creation.

Changed retry

Add required OUTPUT_DIR and optional LOG_LEVEL with explicit defaults/validation.

A14 - DE05-14 | Logging, testing and reusable pipeline code

One acceptable solved answer

Write focused tests for invalid date and duplicate-ID policy. Break exactly one rule, observe the failing test, repair the rule, and rerun. Logs should carry run/context/error information without secrets.

Worked proof / example

```
def test_invalid_date_rejected():
    ok, reason=validate({'date':'31/99/2026','id':'A1'})
```

```
assert not ok and reason=='invalid_date'

def test_duplicate_id_policy():
    assert split(rows_with_duplicate).rejected_count==2
```

Common beginner mistakes

Tests that only assert “no exception”; mixing multiple defects into one test; logging whole headers/tokens.

Validate your own attempt

Show red -> repair -> green for each focused test and preserve the original failed output.

Changed retry

Add a missing-required-field test and make the failure reason deterministic.

A15 - DE05-15 | Integrated Python data-engineering pipeline

One acceptable solved answer

A complete solution must preserve raw evidence, validate deterministically, separate rejects, reconcile counts/totals, write target output only after validation, and save run evidence. The fresh checkpoint must be solved with review closed.

Worked proof / example

Minimum evidence: source hash + raw count; accepted/rejected counts and reasons; accepted business total; target row/key controls; one deliberate failure/recovery note; rerun/replay result.

A plausible control equation is raw rows = accepted rows + rejected rows; target count/key set must equal accepted rows after any intended aggregation/dedup rule.

Common beginner mistakes

Copying the guided implementation into the checkpoint; accepting a final CSV merely because it exists; advancing state after failed validation.

Validate your own attempt

Run from a clean working directory and compare independent counts/key sets/totals to the saved run evidence.

Changed retry

Change one invalid row class and one business amount, then rerun from raw input and explain the changed reconciliation.